

# The Impact of Loop Unrolling on Controller Delay in High-Level Synthesis

Christian Percival  
Hochschule Hamm-Lippstadt  
Electronic Engineering  
Email: percival.christian@gmail.com

**Abstract**—Loop unrolling is a widely used optimization technique that can improve performance by exposing parallelism and reducing loop-control overhead. In high-level synthesis (HLS), however, loop unrolling has hardware-specific effects that are not always visible at the software level. When a high-level program is transformed into hardware, the loop body, datapath, controller, finite-state machine (FSM), multiplexers, and resource-sharing logic can all be affected by the selected unroll factor. This paper studies the impact of loop unrolling on controller delay in HLS, based mainly on the work of Kurra, Singh, and Panda. The central idea is that larger unroll factors can reduce loop latency, but they can also increase controller complexity and timing pressure. Therefore, the best unroll factor is not necessarily the largest possible unroll factor. In addition to explaining the reference paper, this seminar work proposes a student-level implementation using a simplified Python estimation model and, if available, an HLS tool such as AMD Vitis HLS, Intel oneAPI FPGA tools, Bambu, or LegUp. The implementation is intended to demonstrate the same tradeoff in a low-budget academic environment by comparing latency, estimated controller delay, and resource-related metrics for different unroll factors.

**Index Terms**—High-Level Synthesis, Loop Unrolling, Controller Delay, Finite-State Machine, Design Space Exploration, FPGA, Embedded Systems

## I. INTRODUCTION

High-level synthesis is a design methodology that transforms an algorithmic or behavioral description, often written in C or C++, into register-transfer level (RTL) hardware. Instead of manually designing every register, multiplexer, arithmetic unit, and state machine, the designer specifies the intended behavior at a higher level of abstraction. The HLS tool then generates a hardware architecture that implements this behavior under given constraints such as clock period, resource usage, and interface requirements.

This approach can significantly improve design productivity. However, it also introduces an important challenge: compiler-style optimizations do not always have the same consequences in hardware as they do in software. One example is loop unrolling. In software compilation, loop unrolling is usually applied to reduce loop overhead and expose more instruction-level parallelism. In HLS, the same transformation can duplicate operations in the generated hardware and increase the size of the controller and datapath.

The main reference paper by Kurra, Singh, and Panda studies exactly this issue. It shows that loop unrolling can improve latency, but it can also increase the delay of the controller

generated during high-level synthesis [1]. If controller delay is ignored, a design that appears efficient at the scheduling level may fail to meet its required clock period after synthesis. Therefore, loop unrolling in HLS must be evaluated as a hardware-aware tradeoff.

The aim of this seminar paper is to explain the technical contribution of the reference paper, place it in the context of HLS and embedded systems, critically evaluate its assumptions and limitations, and propose a student-level implementation that can demonstrate the same principle using accessible tools.

The research question of this paper is:

How does loop unrolling affect the tradeoff between latency, controller complexity, and timing feasibility in high-level synthesis, and how can this effect be demonstrated in a low-budget student implementation?

## II. TECHNICAL BACKGROUND

### A. High-Level Synthesis

High-level synthesis is an automated design process that converts a high-level behavioral specification into RTL hardware. A typical HLS flow starts with a C or C++ function, verifies the behavior using software simulation, generates RTL for a target clock period, and then verifies the generated RTL using co-simulation or traditional hardware verification methods [7].

The generated hardware usually consists of two main parts:

- a datapath, which contains functional units, registers, memories, and multiplexers;
- a controller, which controls the execution order and resource selection.

The controller is often implemented as a finite-state machine. It decides which operations are active in each clock cycle and which datapath resources are selected. This means that the controller is not only a logical scheduling element, but also a physical hardware structure that can contribute to critical path delay.

### B. Loop Unrolling

Loop unrolling duplicates the body of a loop so that several iterations are represented together. A simple loop such as:

```
for (int i = 0; i < 32; i++) {  
    q += x[i] * z[i];  
}
```

can be partially unrolled by a factor of two:

```
for (int i = 0; i < 32; i += 2) {
    q += x[i] * z[i];
    q += x[i+1] * z[i+1];
}
```

The unrolled version performs two original loop iterations inside one new loop iteration. This can reduce loop-control overhead and expose additional parallelism. In HLS, the additional operations may be scheduled in parallel if enough hardware resources and memory bandwidth are available.

However, unrolling also increases the size of the loop body. This can increase the number of operations that must be scheduled and controlled. It can also increase the size of multiplexers when several operations share the same functional units. Therefore, the hardware effect of loop unrolling is more complex than the software effect.

### C. Controller Delay

The reference paper explains that the critical timing path of a synthesized design may include the controller, multiplexers, function units, and registers [1]. If loop unrolling increases the complexity of the controller, then the controller delay may increase. This can reduce the maximum clock frequency of the final design.

Controller delay is especially important in HLS because the generated controller is determined by the scheduling and binding decisions made by the synthesis tool. Larger unroll factors can increase:

- the number of operations in the data-flow graph;
- the number of scheduled control steps;
- the number of FSM states;
- the number of state bits;
- the complexity of resource sharing;
- the size of multiplexers.

Therefore, an HLS schedule that reduces latency in cycles may still be problematic if it creates a longer clock period.

## III. MAIN REFERENCE PAPER

### A. Problem Addressed

Kurra, Singh, and Panda investigate the impact of loop unrolling on controller delay in HLS [1]. Their motivation is that loop unrolling is a common compiler optimization, but its hardware consequences in HLS are not straightforward. While unrolling can improve performance by reducing loop iterations and exposing parallelism, it can also increase the complexity and delay of the generated controller.

The paper argues that unroll factor selection should consider not only latency but also FSM delay. The authors propose an estimation-based method to predict controller delay as a function of the loop unroll factor. This prediction is then used to guide design-space exploration without exhaustively synthesizing every possible unroll factor.

### B. Loop Unrolling Example

The reference paper uses an example loop that accumulates products from two arrays. The original loop has 32 iterations. With given resource constraints, the original loop requires approximately 128 cycles. When the loop is unrolled once, corresponding to an unroll factor of two, the loop requires approximately 96 cycles [1]. This shows a performance improvement.

However, the paper also shows that the unrolled version has more control states than the original version. The original schedule has four states, while the unrolled schedule has six states. This illustrates the central tradeoff: the unrolled design can reduce latency but increase FSM complexity.

### C. Delay Estimation Model

The paper uses a high-level estimation model for controller delay. A simplified form of the model is:

$$Delay = A \cdot \log(\#operations) \cdot (\#state\ bits) \quad (1)$$

where  $A$  is a technology-dependent factor influenced by aspects such as functional-unit control bits and the number of variables [1]. The number of operations increases when the loop body is duplicated. The number of state bits depends on the number of states or control steps required by the schedule.

This model is useful because it allows delay to be estimated from high-level information instead of requiring full synthesis for every candidate design. The estimation is not intended to replace final synthesis, but to guide the exploration process.

### D. Search Space Pruning

The paper observes that latency does not always improve monotonically with the unroll factor. If the unroll factor does not evenly divide the loop bound, a trailer loop may be required for the remaining iterations. This can reduce the expected benefit of unrolling.

The paper also observes that moderate unroll factors often provide most of the latency benefit while avoiding the worst delay increase. Therefore, the authors reduce the design-space exploration problem by focusing on promising unroll factors rather than exhaustively testing all possibilities.

### E. Priority Function

To select useful unroll factors, the paper uses a priority function based on the ratio between latency reduction and additional FSM delay. A simplified version is:

$$Priority = \frac{Latency\ Reduction}{Additional\ FSM\ Delay} \quad (2)$$

This is similar to a value-density heuristic. A loop is more attractive for unrolling if it provides a large latency reduction for a small additional delay cost. This approach helps distribute the available delay budget among loops rather than spending it entirely on one aggressive unrolling decision.

#### IV. SCIENTIFIC CONTEXT AND RELATED WORK

##### A. Loop Unrolling in Compiler Optimization

Loop unrolling is well established in compiler optimization. Classical work on loop unrolling focuses on improving processor performance by reducing loop overhead, improving instruction scheduling, and increasing the ratio of computation to memory operations. Sarkar studied optimized unrolling of nested loops [2], while Carr and Kennedy investigated improving the ratio of memory operations to floating-point operations in loops [3]. Koseki et al. considered methods for estimating optimal unrolling times for nested loops [4].

These works are relevant because they provide the compiler background for loop unrolling. However, they do not directly address the HLS-specific problem of controller delay. In software, unrolling increases code size and can affect cache behavior. In HLS, unrolling can physically change the generated hardware architecture.

##### B. Timing and Control Delay in HLS

The reference paper builds on earlier work related to control delay estimation in high-level synthesis. Gupta, Gupta, and Panda proposed rapid estimation of control delay from high-level specifications [5]. This is directly relevant because Kurra et al. use a similar idea to estimate FSM delay caused by different loop unroll factors.

Software pipelining is also related because it improves loop performance by overlapping iterations. Allan et al. provide a broad survey of software pipelining techniques [6]. In modern HLS, loop pipelining and loop unrolling are often used together or compared against each other. However, both techniques can affect resources, scheduling, and timing.

##### C. Modern HLS Tool Context

Modern HLS tools make loop optimization accessible through directives and pragmas. AMD Vitis HLS includes support for loop pipelining and loop unrolling, and provides C simulation, C synthesis, and C/RTL co-simulation flows [7]. Intel's oneAPI FPGA Handbook also includes loop analysis, loop optimization, FPGA reports, unroll pragmas, and timing/resource analysis for FPGA designs [8].

Open-source or academic HLS frameworks are also relevant for student-level implementation. Bambu is described as a free framework for assisting high-level synthesis of complex applications [9]. LegUp provides an HLS flow with documentation on high-level synthesis concepts, loop-level parallelism, loop pipelining, memory partitioning, and software/hardware co-simulation [10].

These tool references are not the main theoretical foundation of the paper. Instead, they are useful for discussing how the concept could be demonstrated in a student implementation.

#### V. TRADEOFF ANALYSIS

##### A. Latency Versus Controller Complexity

The main tradeoff is between reducing latency and increasing controller complexity. A small unroll factor keeps the

controller simpler but may leave much of the loop latency unchanged. A medium unroll factor can often provide a strong latency improvement while keeping resource usage and controller complexity manageable. A large or full unroll factor exposes the maximum amount of parallelism but may produce excessive area and timing pressure.

This means that the optimal unroll factor depends on the design constraints. If the design is latency-critical and sufficient hardware resources are available, a larger unroll factor may be useful. If the design is area-constrained or timing-constrained, a smaller unroll factor may be more practical.

##### B. Cycle Latency Versus Clock Period

A design with fewer cycles is not automatically faster. The total execution time depends on both the number of cycles and the clock period:

$$\text{Execution Time} = \text{Latency in Cycles} \times \text{Clock Period} \quad (3)$$

Loop unrolling can reduce the number of cycles. However, if it increases the critical path and forces a longer clock period, the final execution time may not improve as expected. This is the key reason why controller delay matters.

##### C. Resource Tradeoffs

Loop unrolling can increase resource usage. Depending on the HLS tool and resource constraints, unrolling may require more:

- adders;
- multipliers;
- DSP blocks;
- registers;
- multiplexers;
- memory ports;
- control signals.

If resources are limited, the HLS tool may share functional units between operations. Resource sharing can reduce area but increase multiplexer and controller complexity. If resources are abundant, the tool may create more parallel hardware, improving latency but increasing area.

##### D. Design Space Exploration

When only one loop is considered, trying several unroll factors may be manageable. However, real applications can contain multiple loops. If each loop has several possible unroll factors, the number of possible design combinations grows quickly. Exhaustive synthesis can become too slow.

The reference paper addresses this by using estimation and pruning. Instead of synthesizing every candidate, it estimates controller delay and focuses on promising factors. This makes the approach more scalable for larger designs.

TABLE I  
TRADEOFF SUMMARY FOR LOOP UNROLLING IN HLS

| Design Choice                | Advantages                      | Disadvantages                                    |
|------------------------------|---------------------------------|--------------------------------------------------|
| No unrolling                 | Simple controller and low area  | Higher latency                                   |
| Partial unrolling            | Balanced latency and complexity | Requires careful factor selection                |
| Full unrolling               | Maximum exposed parallelism     | High resource usage and possible timing problems |
| Exhaustive exploration       | Can find strong candidates      | Very high synthesis time                         |
| Estimation-based exploration | Faster and more scalable        | Less exact than full synthesis                   |

## VI. CRITICAL EVALUATION

### A. Strengths

The main strength of the reference paper is that it highlights a practical HLS problem that can easily be overlooked. Loop unrolling is often introduced as a performance optimization, but the paper shows that it can also increase controller delay. This helps connect high-level program transformations to low-level hardware timing effects.

A second strength is the use of estimation-based design-space exploration. Instead of requiring full synthesis for every possible unroll factor, the paper estimates delay from high-level information. This can significantly reduce exploration time.

A third strength is that the paper does not treat latency as the only metric. It considers the timing feasibility of the generated hardware, which is important for embedded systems and FPGA or ASIC accelerators.

### B. Limitations

One limitation is that the delay model depends on the synthesis flow and target technology. The constants used in the delay model are not universal. They must be calibrated for a specific synthesis environment.

Another limitation is that the original experiments were performed using a specific behavioral synthesis framework and ASIC library. The exact numerical results may not directly transfer to modern FPGA-based HLS tools.

Modern HLS tools may also perform additional optimizations automatically. These optimizations can make it difficult to isolate controller delay from datapath delay, routing delay, memory effects, and tool-specific scheduling decisions.

### C. Applicability

The approach is most applicable to loop-based hardware accelerators where the loop structure and bounds are known. Suitable examples include dot-product accelerators, FIR filters, simple image-processing kernels, matrix operations, and embedded signal-processing loops.

The approach is less suitable for highly irregular programs, loops with unknown bounds, or designs where memory access patterns dominate performance. It may also be less useful for very small loops where exhaustive exploration is already cheap.

## VII. PROPOSED STUDENT IMPLEMENTATION

### A. Implementation Objective

The goal of the implementation is to demonstrate the main tradeoff discussed in the reference paper using a low-budget student environment. The implementation does not aim to reproduce the original ASIC synthesis flow exactly. Instead, it aims to show the same conceptual relationship:

- increasing the unroll factor can reduce loop latency;
- increasing the unroll factor can increase operation count and controller complexity;
- the best unroll factor is therefore a tradeoff.

The implementation will be presented as a practical learning experiment. It can be done first as a Python simulation and later extended with an HLS tool if available.

### B. Benchmark Kernel

The planned benchmark is a simple dot-product loop:

```
#define N 32

int dot_product(int x[N], int z[N]) {
    int q = 0;

    for (int i = 0; i < N; i++) {
        q += x[i] * z[i];
    }

    return q;
}
```

This benchmark is appropriate because it is simple, loop-based, and similar to the example used in the reference paper. It contains repeated memory access, multiplication, and accumulation.

### C. Unroll Factors

The following unroll factors are planned:

$$u \in \{1, 2, 4, 8, 16, 32\} \quad (4)$$

The factor  $u = 1$  represents the original rolled loop. The factor  $u = 32$  represents full unrolling for a loop bound of 32.

### D. Python Estimation Model

The first implementation will be a Python-based estimation model. This model will not generate real hardware, but it will demonstrate the trend described in the reference paper.

The model will estimate:

- number of loop iterations after unrolling;
- operation count;
- estimated loop-body latency;
- estimated total latency;
- estimated number of states;
- number of state bits;
- estimated controller delay.

A simplified estimation approach is:

$$\text{Iterations} = \left\lceil \frac{N}{u} \right\rceil \quad (5)$$

$$OperationCount = BaseOps + (u - 1) \cdot LoopBodyOps \quad (6)$$

$$BodyLatency = OriginalBodyLatency + (u - 1) \cdot II \quad (7)$$

$$TotalLatency = Iterations \cdot BodyLatency \quad (8)$$

$$StateBits = \lceil \log_2(States) \rceil \quad (9)$$

$$ControllerDelay = A \cdot \log_2(OperationCount) \cdot StateBits \quad (10)$$

Here,  $A$  is not the calibrated ASIC constant from the reference paper. Instead, it is a student-level demonstration constant used to show the expected trend.

#### E. Planned Python Code

The following Python script is planned for the first implementation:

```
import math
import pandas as pd
import matplotlib.pyplot as plt

N = 32
unroll_factors = [1, 2, 4, 8, 16, 32]

loop_body_ops = 4
base_ops = 4

original_body_latency = 4
initiation_interval = 2

A = 0.35

results = []

for u in unroll_factors:
    iterations = math.ceil(N / u)

    operation_count = base_ops + (u - 1) *
        loop_body_ops

    body_latency = original_body_latency + (u - 1) *
        initiation_interval

    estimated_latency = iterations * body_latency

    estimated_states = max(body_latency, 2)
    state_bits = math.ceil(math.log2(estimated_states))

    controller_delay = A * math.log2(operation_count) *
        state_bits

    results.append({
        "unroll_factor": u,
        "iterations": iterations,
        "operation_count": operation_count,
        "body_latency": body_latency,
        "estimated_latency": estimated_latency,
        "estimated_states": estimated_states,
        "state_bits": state_bits,
        "estimated_controller_delay": round(
            controller_delay, 3)
    })
```

```
)

df = pd.DataFrame(results)
print(df)

df.to_csv("loop_unrolling_results.csv", index=False)

plt.figure()
plt.plot(df["unroll_factor"], df["estimated_latency"],
    marker="o")
plt.xlabel("Unroll_factor")
plt.ylabel("Estimated_latency_in_cycles")
plt.title("Estimated_latency_vs_loop_unroll_factor")
plt.grid(True)
plt.savefig("latency_vs_unroll_factor.png", dpi=300)
plt.show()

plt.figure()
plt.plot(df["unroll_factor"], df["estimated_controller_delay"],
    marker="o")
plt.xlabel("Unroll_factor")
plt.ylabel("Estimated_controller_delay")
plt.title("Estimated_controller_delay_vs_loop_unroll_factor")
plt.grid(True)
plt.savefig("controller_delay_vs_unroll_factor.png",
    dpi=300)
plt.show()
```

#### F. Planned Result Table

After running the Python model, the results will be inserted into a table similar to Table II. The current table is a placeholder and will be replaced with measured or generated values.

TABLE II  
PLANNED RESULT TABLE FOR PYTHON SIMULATION

| Unroll | Iterations | Ops | Latency | Ctrl. Delay |
|--------|------------|-----|---------|-------------|
| 1      | TBD        | TBD | TBD     | TBD         |
| 2      | TBD        | TBD | TBD     | TBD         |
| 4      | TBD        | TBD | TBD     | TBD         |
| 8      | TBD        | TBD | TBD     | TBD         |
| 16     | TBD        | TBD | TBD     | TBD         |
| 32     | TBD        | TBD | TBD     | TBD         |

#### G. Expected Graphs

The implementation should produce at least two graphs:

- unroll factor versus estimated latency;
- unroll factor versus estimated controller delay.

The expected trend is that estimated latency decreases as the unroll factor increases, while estimated controller delay increases. This would support the main argument that unrolling should not be selected only by looking at latency.

#### H. Optional HLS Implementation

If time and tools are available, the Python model can be extended with an HLS experiment. The same dot-product function can be synthesized multiple times with different unroll factors.

In AMD Vitis HLS, the unroll directive could be applied as follows:

```

int dot_product(int x[32], int z[32]) {
    int q = 0;

    loop_main:
    for (int i = 0; i < 32; i++) {
        #pragma HLS UNROLL factor=4
        q += x[i] * z[i];
    }

    return q;
}

```

The factor would be changed for each experiment. The Vitis HLS flow can then be used to run C simulation, C synthesis, and possibly C/RTL co-simulation [7].

For each unroll factor, the following values could be recorded from the HLS reports:

- latency in cycles;
- initiation interval, if pipelining is used;
- estimated clock period;
- timing slack;
- LUT usage;
- flip-flop/register usage;
- DSP usage;
- BRAM usage;
- synthesis time.

Intel oneAPI FPGA tools could also be used because the oneAPI FPGA Handbook includes loop analysis, loop optimization, unrolling, and report analysis features [8]. Bambu and LegUp are possible alternatives for an open-source or academic HLS context [9], [10].

### I. Limitations of the Implementation

The Python simulation is only an estimation model. It does not generate RTL and cannot prove real hardware timing. Its purpose is to demonstrate the concept clearly and produce data for discussion.

The optional HLS experiment would be more realistic, but it may still not isolate controller delay directly. Many modern tools report total timing estimates rather than a separate FSM delay. Therefore, timing slack, clock estimate, and resource usage may need to be used as indirect evidence.

The implementation should therefore be presented honestly as a student-level demonstration, not as a complete reproduction of the original paper.

## VIII. DISCUSSION

The planned implementation supports the educational goal of the seminar because it connects the theory from the reference paper to a practical experiment. Even a simplified Python model can make the tradeoff visible. It allows the student to explain why larger unroll factors reduce the number of loop iterations but increase operation count and estimated controller delay.

If an HLS experiment is completed, the paper can include real tool-generated reports. This would make the study stronger because it would show how modern HLS tools report latency, resources, and timing for different unroll factors.

However, the main contribution of the student implementation is not exact reproduction. The contribution is demonstrating the principle in a simple and understandable way.

The paper also shows that HLS optimization is not only a software problem. The designer must consider the generated hardware structure. A transformation that improves latency in cycles may still be problematic if it increases controller delay or prevents timing closure.

## IX. CONCLUSION

Loop unrolling is an important optimization in high-level synthesis, but it must be applied carefully. The reference paper by Kurra, Singh, and Panda shows that loop unrolling can reduce latency while increasing controller delay [1]. This means that the largest unroll factor is not always the best choice.

The main lesson is that HLS design requires hardware-aware reasoning. Loop unrolling affects not only the number of loop iterations, but also the generated FSM, datapath, multiplexers, and timing behavior. Therefore, unroll factor selection should consider both performance and timing feasibility.

The proposed student implementation will demonstrate this tradeoff using a simplified Python estimation model and, if possible, an HLS tool experiment. The planned results will include a table and graphs comparing unroll factor, estimated latency, operation count, and controller delay. This will provide practical evidence for the central idea of the seminar paper.

Future work could include running the same benchmark in Vitis HLS, Intel oneAPI FPGA tools, Bambu, or LegUp; comparing loop unrolling with loop pipelining; and testing additional kernels such as FIR filters or vector operations.

## REFERENCES

- [1] S. Kurra, N. K. Singh, and P. R. Panda, "The Impact of Loop Unrolling on Controller Delay in High Level Synthesis," in *Design, Automation and Test in Europe Conference and Exhibition (DATE)*, 2007.
- [2] V. Sarkar, "Optimized Unrolling of Nested Loops," in *Proceedings of the 14th International Conference on Supercomputing*, 2000.
- [3] S. Carr and K. Kennedy, "Improving the Ratio of Memory Operations to Floating-Point Operations in Loops," *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 6, 1994.
- [4] A. Koseki et al., "A Method for Estimating Optimal Unrolling Times for Nested Loops," in *International Symposium on Parallel Architectures, Algorithms, and Networks*, 1997.
- [5] G. R. Gupta, M. Gupta, and P. R. Panda, "Rapid Estimation of Control Delay from High-Level Specifications," in *Design Automation Conference (DAC)*, pp. 455–458, 2006.
- [6] V. H. Allan, R. B. Jones, R. M. Lee, and S. J. Allan, "Software Pipelining," *ACM Computing Surveys*, vol. 27, no. 3, pp. 367–432, 1995.
- [7] AMD, *Vitis High-Level Synthesis User Guide, UG1399, v2025.2*, Jan. 2026.
- [8] Intel, *Intel oneAPI FPGA Handbook*, 2024.
- [9] Politecnico di Milano, "Bambu HLS Framework," Panda Project.
- [10] Microchip Technology Inc., *LegUp 2021.1 Documentation: User Guide*, 2021.